

Универзитет Унион
Рачунарски Факултет
Рачунарске науке

ДИПЛОМСКИ РАД

Дигитална каса за угоститељске објекте: пројектовање и имплементација веб система са више закупаца

Кандидат:
Игор Гајић
РН 78/2023

Ментор:
Професор Душан Вујошевић

Београд, 2026.

Апстракт

У раду је пројектован и имплементиран веб систем који замењује папирну евиденцију пословања у угоститељском објекту, и то тако да једна инсталација опслужује више међусобно одвојених објеката. Раздвајање података изведено је на нивоу контекста приступа бази, а не појединачног упита, чиме се могућност да један објекат види туђе податке своди на грешку у једној класи уместо на грешку у било ком од неколико десетина упита.

Уведена је мерена величина која описује услугу а не новац: време од тренутка када је поруџбина примљена до тренутка када је изнесена, по запосленом и по периоду, са извозом у табеларни формат. Екран сале освежава се сам, без поновног учитавања странице, и пропраћен је кратким звучним сигналом, јер се у бучном објекту у екран по правилу нико не гледа.

Гост може да поручи скенирањем кода са свог стола, без налога и без инсталације апликације, при чему токен из кода доказује искључиво о ком је столу реч. Контрола приступа описана је на једном месту, у облику матрице улога и овлашћења, коју клијентска апликација пресликава како кориснику не би нудила екран који би сервер одбио.

Исправност је проверавана на четири нивоа, од јединичних тестова доменских правила до проласка кроз сваку руту на живој бази. Последња два нивоа настала су тек након што се показало да поједине грешке остају невидљиве за тестове који не разговарају са правим сервером базе података.

Кључне речи

Кључне речи су изабране тако да опишу и домен примене и техничке одлуке које рад образлаже. Поређане су од ширег појма ка ужем.

дигитална каса, угоститељство, веб апликација, више закупаца, чиста архитектура, раздвајање команди и упита, контрола приступа заснована на улогама, комуникација у реалном времену, извештавање, .NET, Angular

Садржај

Апстракт	2
Кључне речи.....	3
1. Увод	7
1.1. Мотивација	7
1.2. Предмет и циљ рада	7
1.3. Оригиналан допринос рада.....	8
1.4. Структура рада.....	8
2. Пословни контекст и преглед постојећих решења	10
2.1. Шта ради каса у угоститељском објекту	10
2.2. Регулаторни оквир у Србији.....	10
2.3. Преглед постојећих решења	11
2.4. Шта из прегледа следи за овај рад	12
3. Поставка проблема и захтеви	13
3.1. Актери.....	13
3.2. Функционални захтеви.....	13
3.3. Нефункционални захтеви	14
3.4. Свесна ограничења обима	14
4. Коришћене технологије	16
4.1. Серверска страна	16
4.2. Клијентска страна.....	16
4.3. База података.....	17
4.4. Образложење избора у целини.....	17
5. Архитектура система.....	18
5.1. Слојевита архитектура	18
5.2. Раздвајање команди и упита.....	19
5.3. Два серверска домаћина.....	20

5.4. Пут једног захтева	20
5.5. Комуникација у реалном времену	21
6. Модел података.....	23
6.1. Раздвајање закупаца	23
6.2. Пословни део модела	24
6.3. Новац као вредносни објекат	25
6.4. Записи који се не бришу	25
7. Безбедност и контрола приступа.....	27
7.1. Аутентификација	27
7.2. Матрица овлашћења.....	27
7.3. Сесија стола.....	27
7.4. Лиценцни зид	28
8. Кориснички интерфејс	29
8.1. Екран сале.....	29
8.2. Поништавање и звучни сигнал.....	30
8.3. Гостов екран.....	30
8.4. Теме.....	31
8.5. Величина првог учитавања.....	33
9. Извештавање	34
9.1. Начело: ниједан извештај не ласка	34
9.2. Промет по данима.....	34
9.3. Сторнирање	35
9.4. Учинак по конобару	35
9.5. Извоз у табеларни формат	36
10. Тестирање и провера	38
10.1. Јединични тестови домена.....	38
10.2. Тестови апликацијског слоја	38

10.3. Интеграциони тестови.....	38
10.4. Пролаз кроз живи систем.....	39
10.5. Резултати	39
11. Критичка дискусија	40
11.1. Критика сопствене поставке.....	40
11.2. Критика стања у области	40
11.3. Шта је могло другачије	41
12. Правци даљег рада.....	42
12.1. Екран за кухињу.....	42
12.2. Делови сале и распоред по њима	42
12.3. Више валута и пореских стопа	42
12.4. Рад без везе са интернетом	42
12.5. Анализа времена чекања по сатима и данима	42
13. Закључак.....	44
Литература	45
Додатак А: изводи из програмског кода	46
А.1. Филтер закупца над упитима.....	46
А.2. Доменско правило: изношење рунде	47
А.3. Приписивање рунде и исечак сати.....	47
А.4. Звучни сигнал.....	48
А.5. Препознавање нове порудбине у реду.....	49

1. Увод

Угоститељски објекат је једно од ретких места на којима се свакодневно рукује новцем, залихама и распоредом људи, а да се при томе често и даље користи папир. Овај рад полази од те разлике између количине података који настају у току једне смене и скромности средстава којима се обично бележе.

Уводно поглавље објашњава одакле потиче избор теме, шта је предмет и циљ рада, у чему је његов оригиналан допринос и како је остатак текста организован. Намера је да читалац после овог поглавља зна шта рад покушава и по чему ће се судити да ли је у томе успео.

1.1. Мотивација

Моја мотивација за ову тему потиче од посматрања свакодневног рада у угоститељским објектима, у којима се велики део евиденције и даље води на папиру или у табелама које нико после не чита. Власник на крају смене зна колико је новца у каси, али обично не зна колико се просечно чекало на поруџбину, који артикал се продаје са најмањом разликом у цени, нити колико је пута нешто сторнирано и зашто.

Уз то ме је занимало да један систем изведем од доменског модела до екрана, а не да напишем још једну апликацију у којој пословна правила живе разбацана по контролерима. Угоститељство се за то показало као захвалан домен, јер има јасна правила која се не смеју прекршити — рачун се не мења пошто је плаћен, залиха се не раздужује двапут, сто се не ослобађа док се не наплати — а та правила су довољно мала да се могу и написати и проверити.

Конечно, тема је одабрана и зато што допушта да се рад провери на нечему видљивом. Систем који ради може се показати, а грешка у њему се примети, што није увек случај са темама у којима се резултат своди на тврдњу.

1.2. Предмет и циљ рада

Предмет рада је пројектовање и имплементација веб система који покрива свакодневно пословање угоститељског објекта: отварање и наплату рачуна, јеловник, залихе, резервације, распоред смена и извештавање. Систем је замишљен

тако да једна инсталација опслужује више објеката који један о другом не знају ништа, што се у литератури назива радом са више закупаца (енгл. multitenancy).

Циљ рада је да покаже како се такав систем може изградити тако да пословна правила остану на једном месту и да се могу проверити независно од оквира (енгл. framework) у ком је апликација написана. Секундарни циљ је да се опишу одлуке које су донете успут, укључујући и оне које су се касније показале као погрешне, јер је управо тај део обично изостављен из документације готових решења.

1.3. Оригиналан допринос рада

Оригиналан допринос рада је урађена имплементација целовитог система за пословање угоститељског објекта са више закупаца. Имплементација је изведена тако да је раздвајање података својство контекста приступа бази а не појединачног упита, уз систематизацију на српском језику појмова и образаца који се у овој области обично срећу само на енглеском.

Уз то, у систем је уведена и мерена величина која описује услугу а не промет — време од пријема до изношења поруџбине, срачунато по запосленом — која се, према ономе што се могло пронаћи у доступним описима сличних решења, ретко нуди као готов извештај. Допринос је и опис начина провере у четири нивоа, у ком су два последња нивоа настала као одговор на конкретне грешке невидљиве за тестове који не разговарају са правим сервером базе података.

1.4. Структура рада

Рад је организован тако да прати пут од проблема ка решењу и, на крају, ка његовој критици. Поглавља се могу читати редом, али су поглавља о архитектури, моделу података и безбедности написана тако да се свако од њих може читати и засебно.

- Поглавље 2 описује пословни контекст и даје преглед постојећих решења.
- Поглавље 3 поставља проблем, актере и захтеве, и именује свесна ограничења обима.
- Поглавље 4 образлаже избор технологија.
- Поглавље 5 описује архитектуру система и пут једног захтева кроз њу.
- Поглавље 6 описује модел података и начин на који су закупци раздвојени.
- Поглавље 7 описује аутентификацију, матрицу овлашћења и лиценцни зид.
- Поглавље 8 описује кориснички интерфејс и одлуке донете у вези са њим.
- Поглавље 9 описује извештавање, укључујући и извештај о учинку по конобару.
- Поглавље 10 описује тестирање и проверу у четири нивоа.

- Поглавља 11, 12 и 13 доносе критичку дискусију, правце даљег рада и закључак.

2. Пословни контекст и преглед постојећих решења

Пре него што се опише решење, корисно је описати посао који оно треба да подржи. Ово поглавље укратко износи шта се у угоститељском објекту заиста дешава током смене, какав је регулаторни оквир у Србији и каква решења већ постоје на тржишту.

2.1. Шта ради каса у угоститељском објекту

Продајно место (енгл. point of sale) у угоститељству разликује се од продајног места у малопродаји по једној особини: рачун се не затвара одмах. Гост седне за сто, поручује у више наврата током неколико сати, и тек на крају плаћа, па је основна јединица евиденције отворени рачун везан за сто, а не појединачна трансакција.

Из те особине следи већина осталих захтева. Сто мора имати стање — слободан, заузет или резервисан — које се види издалека; рачун мора моћи да прими нову ставку сатима након отварања; а цена ставке мора остати она која је важила у тренутку поручивања, јер би иначе измена јеловника преписала историју свих отворених рачуна.

Уз то иде и неколико споредних токова који се у пракси покажу као неизбежни. Нешто се укуца грешком па се мора сторнирати, гост одустане од поруџбине, конобар однесе пиће погрешном столу, а смена се мења усред сервиса — и сваки од тих случајева оставља траг који власник после тражи.

2.2. Регулаторни оквир у Србији

Издавање фискалних рачуна у Републици Србији уређено је Законом о фискализацији [11], који прописује да се промет на мало евидентира преко електронског фискалног уређаја и да издати рачун мора бити прослеђен Пореској управи. Електронски фискални уређај се, према том оквиру, састоји од процесора фискалних рачуна и електронског система за издавање рачуна, при чему процесор мора бити одобрен од стране надлежног органа.

За рад оваквог обима то значи једну важну границу. Систем описан у овом раду издаје отисак рачуна који изгледом подсећа на фискални, али који није прошао ни кроз један одобрен процесор фискалних рачуна, па је на самом отиску то и назначено; свака евентуална стварна примена захтевала би интеграцију са

одобренем уређајем и вероватно додатну проверу усклађености, што излази из оквира дипломског рада.

Поред фискализације, на систем овог типа односи се и заштита података о личности, будући да се у бази чувају имена запослених и, у случају резервација, контакт подаци гостију. Рад се тиме не бави детаљно, али је гостов екран намерно пројектован тако да не тражи никакве личне податке, чиме се, изгледа, највећи део тог питања заобилази већ на нивоу пројектовања.

2.3. Преглед постојећих решења

На тржишту постоји већи број решења за угоститељство, од домаћих програма који се инсталирају на један рачунар у локалу до страних платформи које раде у облаку (енгл. cloud) и наплаћују се месечно. Поређење које следи засновано је на јавно доступним описима производа и на особинама које су за овај рад релевантне, а не на систематском испитивању сваког од њих.

Особина	Класична локална решења	Платформе у облаку	Систем из овог рада
Инсталација	На рачунару у објекту	Без инсталације, у прегледачу	Без инсталације, у прегледачу
Више објеката	По правилу једна по објекту	Подржано, уз одговарајући пакет	Подржано, једна инсталација
Фискализација	Најчешће подржана	Зависи од тржишта	Није подржана, отисак је симулација
Поручивање преко кода са стола	Ретко	Све чешће	Подржано, без налога и без апликације
Мерење времена услуге	Није уобичајено	Није уобичајено	Подржано, по запосленом
Цена	Једнократна лиценца	Месечна претплата	Лиценца по објекту, са зидом при истеку

Табела 1. Поређење приступа према особинама релевантним за овај рад

Из табеле 1 види се да се систем описан у овом раду по већини особина сврстава уз платформе у облаку, али са две разлике. Прва је изостанак фискализације, која је свесно остављена ван обима, а друга је мерење времена услуге, које се у доступним описима других решења ретко среће као готов извештај.

2.4. Шта из прегледа следи за овај рад

Преглед показује да основне функције касе одавно нису истраживачко питање и да би покушај да се у њима нађе оригиналност вероватно био неуспешан. Простор који остаје није у томе шта систем ради, него у томе како је изграђен и шта мери.

Отуда су у овом раду тежиште добиле три ствари: начин на који се раздвајају подаци различитих објеката, начин на који се пословна правила држе одвојена од оквира, и увођење једне мерене величине која описује услугу уместо промета. Остале функције су имплементиране зато што без њих систем не би био употребљив, али се у тексту описују краће.

3. Поставка проблема и захтеви

Захтеви су овде изнети у облику у ком су и настали, то јест полазећи од тога ко систем користи и шта му треба. Подела на функционалне и нефункционалне захтеве служи да се раздвоје особине које се виде на екрану од оних које се примете тек када изостану.

3.1. Актери

Систем препознаје четири врсте корисника, од којих три имају налог, а једна нема. Улоге су хијерархијске у смислу да свака виша улога може све што може нижа, уз изузетке који су наведени у поглављу 7.

Актер	Има налог	Шта ради
Конобар	Да	Отвара и наплаћује рачуне, додаје ставке, прима резервације, износи поруџбине поручене преко кода са стола.
Менаџер	Да	Све што и конобар, уз јеловник, магацин, распоред смена и одобравање сторна плаћеног рачуна.
Власник	Да	Све што и менаџер, уз извештаје, распоред столова, запослене и подешавања објекта.
Гост	Не	Скенира код са свог стола и поручује; не плаћа преко система и не види ништа осим јеловника и онога што је тај сто поручио.
Администратор платформе	Да	Региструје објекте и води лиценце; ради у засебној апликацији и не види промет појединачног објекта.

Табела 2. Актери система и њихова задужења

Администратор платформе из табеле 2 разликује се од осталих по томе што не припада ниједном објекту. Та разлика је касније обликовала архитектуру, јер је захтевала да постоји начин да се намерно заобиђе филтер који све остале држи унутар њиховог објекта.

3.2. Функционални захтеви

Функционални захтеви су груписани по екранима, јер је тако и рађено. Редослед у наставку приближно одговара редоследу имплементације.

1. Пријава на објект шифром објекта, електронском поштом и лозинком, при чему иста адреса електронске поште може постојати у више објеката.
2. Приказ сале са распоредом столова, њиховим стањем и износом отвореног рачуна.
3. Отварање рачуна за сто, додавање ставки, сторнирање ставке и наплата.

4. Отисак рачуна са свим ставкама, начином плаћања и напоменом да није фискални.
5. Вођење јеловника са категоријама, ценама и доступношћу артикала.
6. Вођење магацина са састојцима, нормативима и аутоматским раздуживањем залиха.
7. Резервације столова са провером преклапања и капацитета.
8. Распоред смена, са шаблонима смена и генерисањем распореда за период.
9. Уређивање распореда просторија, столова и елемената попут шанка и тоалета.
10. Поручивање од стране госта скенирањем кода са стола, без налога.
11. Ред поруџбина за изношење, са потврдом изношења и могућношћу поништења.
12. Извештаји о промету, најпродаванијим артиклима, сторнирању и учинку по конобару.
13. Извоз извештаја о учинку по конобару у табеларни формат.
14. Регистрација објеката и вођење лиценци у засебној апликацији.

3.3. Нефункционални захтеви

Нефункционални захтеви су формулисани тако да се могу проверити, јер захтев који се не може проверити у пракси не обавезује ни на шта. Уз сваки је наведено и како је проверен.

Захтев	Како је проверен
Објекат ни под којим условом не види податке другог објекта.	Интеграциони тестови који се пријављују као један објекат и траже туђи запис.
Пословна правила су проверљива без базе и без веб сервера.	Јединични тестови доменског слоја, који не додирују инфраструктуру.
Екран сале се освежава без поновног учитавања странице.	Ручна провера са два отворена прегледача и поруџбином са телефона.
Објекат са истеклом лиценцом не може да ради.	Интеграциони тест који очекује одговор 402 на сваки позив.
Боје стања стола разликују се на свакој од понуђених тема.	Провера контраста и визуелна провера све четири теме.
Интерфејс је на српском језику, са исправном множином.	Јединични тестови функција за обликовање текста.

Табела 3. Нефункционални захтеви и начин њихове провере

Захтев из првог реда табеле 3 је једини који је у току рада третиран као апсолутан. Сви остали допуштају компромис, док би пропуштање овог значило да систем у облику са више закупаца уопште није употребљив.

3.4. Свесна ограничења обима

Неколико ствари је намерно остављено ван обима, и то је забележено пре почетка рада да се одлука не би преиспитивала усред имплементације. Наведене су овде да би се разликовале од пропуста.

- Фискализација, из разлога описаних у одељку 2.2.
- Обрада плаћања картицом преко стварног терминала; начин плаћања се бележи, али се новац не преноси.
- Рад без везе са интернетом, који би захтевао локалну базу и разрешавање сукоба.
- Мобилна апликација за конобаре; систем ради у прегледачу, укључујући и на телефону.
- Екран за кухињу, иако је највећи део механизма за њега већ имплементиран; о томе више у поглављу 12.

Последња ставка заслужује кратко образложење, јер се разликује од осталих. Њу није спречила сложеност него расположиво време, па је описана међу правцима даљег рада, а не међу стварима које систем начелно не ради.

4. Коришћене технологије

Избор технологија је за рад овог типа мање занимљив од начина на који су употребљене, али га треба образложити јер је обликовао неке касније одлуке. У наставку је за сваки слој наведено шта је изабрано и зашто, са напоменом о алтернативи која је разматрана.

4.1. Серверска страна

Серверски део написан је у окружењу .NET 10, на језику C#. Избор је пао на то окружење због статичке типизације, која у домену са новцем и количинама спречава читав низ грешака још при превођењу, и због тога што оно у стандардној испоруци доноси и посредника захтева, и систем аутентификације, и подршку за комуникацију у реалном времену [5].

За приступ бази коришћен је алат за објектно-релационо мапирање (енгл. object-relational mapping) Entity Framework Core [6]. Пресудна је била његова могућност глобалног филтера над упитима, која је, како је описано у одељку 6.1, постала носилац раздвајања закупаца; без ње би се исто морало постизати понављањем услова у сваком упиту.

За раздвајање команди и упита коришћена је библиотека МедијатоР (енгл. MediatR), и то намерно у верзији 12.5.0, која је последња објављена под дозволом Apache 2.0. Касније верзије прелазе на комерцијалну дозволу, што за дипломски рад није препрека, али би за било какву даљу употребу било, па је одлука забележена и у самом пројектном фајлу.

4.2. Клијентска страна

Клијентски део написан је у оквиру Angular, верзија 21, уз библиотеку компоненти Angular Material [8]. Одлучујуће је било то што Angular доноси усмеривач (енгл. router), обрасце и валидацију у самом оквиру, па се за систем ове величине не мора састављати скуп независних библиотека.

Коришћен је радни простор (енгл. workspace) са три пројекта: каса, администрација платформе и заједничка библиотека. Заједничка библиотека је касније подељена на три улазне тачке, из разлога који је описан у одељку 8.5, јер је у првобитном облику непотребно оптерећивала прво читавање апликације.

4.3. База података

Као систем за управљање базом података коришћен је Microsoft SQL Server, у издању Express. Избор је делом практичан, јер се то издање бесплатно користи и добро се слаже са изабраним алатом за мапирање, а делом садржински, јер систем ослања део интегритета на јединствена ограничења и трансакције.

Шема се не одржава ручно него се изводи из доменског модела, миграцијама. Оба серверска домаћина при покретању примењују миграције које недостају, чиме се избегава размимоилажење између кода и базе које се иначе открива тек у тренутку прве грешке.

4.4. Образложење избора у целини

Све три одлуке имају заједничку нит: изабрани су алати који допуштају да пословна правила остану на једном месту и да се могу проверити без покретања остатка система. То је и разлог због ког доменски слој у овом раду нема ниједну спољашњу референцу, па се његови тестови извршавају за око двеста милисекунди.

Разматрана је и алтернатива у којој би серверски део био написан у окружењу Node.js, а клијентски у библиотеци React. Та комбинација вероватно би дала бржи почетак, али би захтевала да се састави и одржава скуп независних библиотека за аутентификацију, валидацију и приступ бази, што за рад у ком се тежиште ставља на архитектуру није деловало као добра размена.

5. Архитектура система

Архитектура је изабрана пре прве линије кода и током рада није мењана, што је вероватно најбољи показатељ да је избор био примерен величини проблема. Ово поглавље описује поделу на слојеве, начин на који су раздвојене команде и упити, разлог постојања два серверска домаћина и пут једног захтева кроз све то.

5.1. Слојевита архитектура

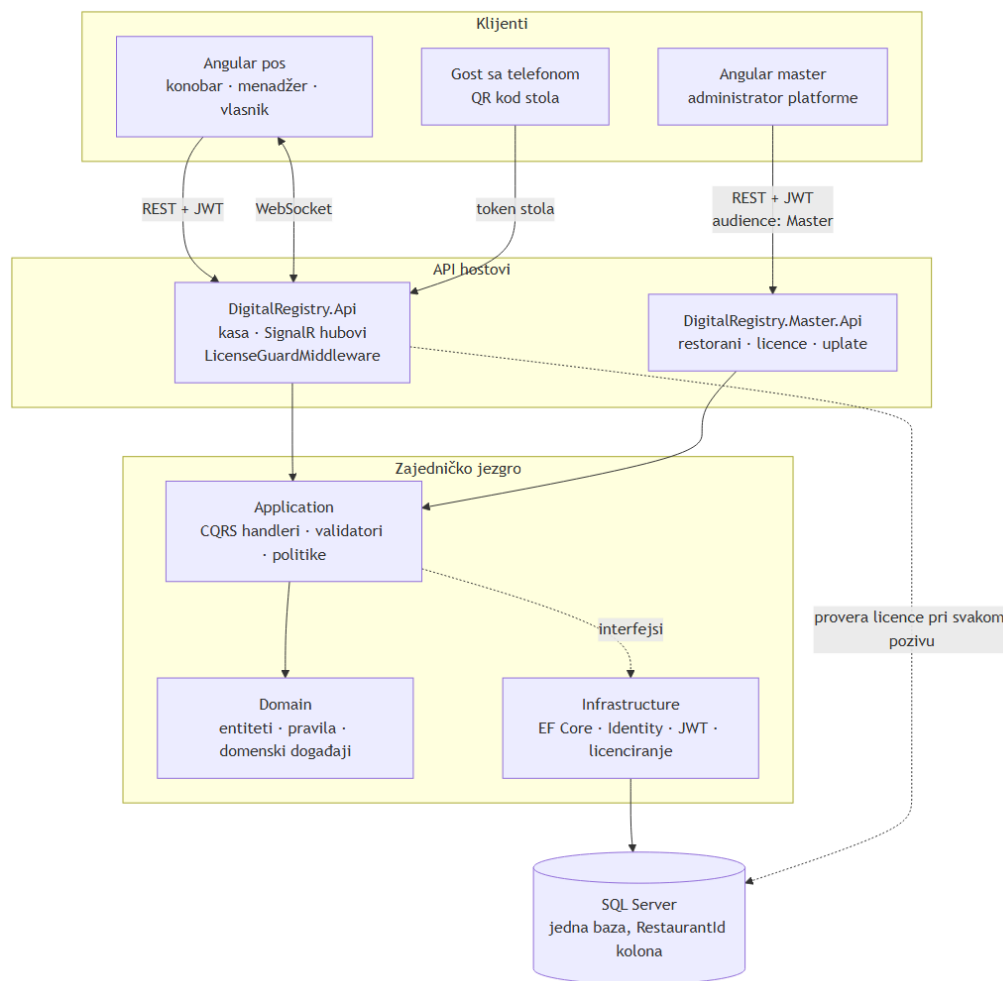
Систем је организован према такозваној чистој архитектури (енгл. clean architecture) [2], чија је основна идеја да зависности показују ка унутра, то јест ка пословним правилима, а никада од њих ка оквиру или бази. Практична последица је да се доменски слој може превести и тестирати без иједне спољне библиотеке.

Подела на четири слоја и правило о томе шта који слој сме да зна дати су у табели 4. Правило је намерно формулисано и у негативном облику, јер се прекршаји лакше примете када је записано шта је забрањено него када је записано само шта је дозвољено.

Слој	Садржи	Не сме да зна
Домен	Ентитете, набрајања, вредносни објекат за новац, доменска правила и догађаје.	Ништа спољашње — нема ниједну референцу.
Апликација	Команде, упите, њихове обрађиваче, валидаторе, објекте за пренос података и матрицу овлашћења.	Алат за мапирање, веб оквир, језик упита над базом.
Инфраструктура	Контекст базе, систем идентитета, издавање жетона, комуникацију у реалном времену, проверу лиценце.	Детаље појединачних екрана.
Серверски домаћини	Контролере, међуслој, регистрацију зависности, конфигурацију.	Пословна правила.

Табела 4. Слојеви система и границе између њих

Целина система приказана је на слици 1. На њој се виде три врсте клијената, два серверска домаћина, заједничко језгро које оба користе и једна база кроз коју све то пролази.



Слика 1. Целина система: клијенти, серверски домаћини, заједничко језгро и база

Испрекидана стрелица од апликацијског ка инфраструктурном слоју на слици 1 иде обрнуто од зависности, и то је намерно. Она означава да апликација декларише интерфејсе, а инфраструктура их имплементира, о чему је више речено у наставку овог одељка.

Зависност апликацијског слоја од инфраструктуре изокренута је увођењем интерфејса. Апликација декларише шта јој треба — приступ бази, идентитет, провера лиценце, контекст закупца — а инфраструктура те интерфејсе имплементира и региструје при покретању, тако да домен и апликација не знају ни за алат за мапирање ни за веб оквир.

5.2. Раздвајање команди и упита

Унутар апликацијског слоја примењен је образац раздвајања одговорности команди и упита (енгл. command query responsibility segregation) [4], у свом блажем облику: команде и упити су одвојени типови са одвојеним обрађивачима, али раде над истим

моделом и истом базом. Пуно раздвајање, са засебним моделом за читање, за систем ове величине не би донело довољно да оправда цену.

Сваки обрађивач прима једну команду или упит и враћа резултат који носи или вредност или разлог неуспеха. Контролери не доносе ниједну одлуку, него само прослеђују захтев посреднику и преводе резултат у одговарајући статусни код, чиме се пословна правила задржавају у слоју у ком се могу тестирати.

Око обрађивача су постављени пресретачи који раде увек исто: један покреће валидацију, други бележи трајање, трећи хвата неочекиване изузетке. Захваљујући томе ниједан обрађивач не садржи проверу улаза, па се у њима чита само пословна логика.

5.3. Два серверска домаћина

Систем има два серверска домаћина над истим доменом и истом базом. Каса ради унутар једног објекта и све што прочита пролази кроз филтер закупца, док администрација платформе ради изнад објекта и тај филтер намерно заобилази.

Подела није козметичка него безбедносна. Да су оба скупа крајњих тачака на истом домаћину, заобилажење филтера било би доступно сваком обрађивачу, а једна грешка у овлашћењима била би довољна да власник једног објекта прочита промет другог; овако је заобилажење филтера ограничено на један директоријум у једном пројекту.

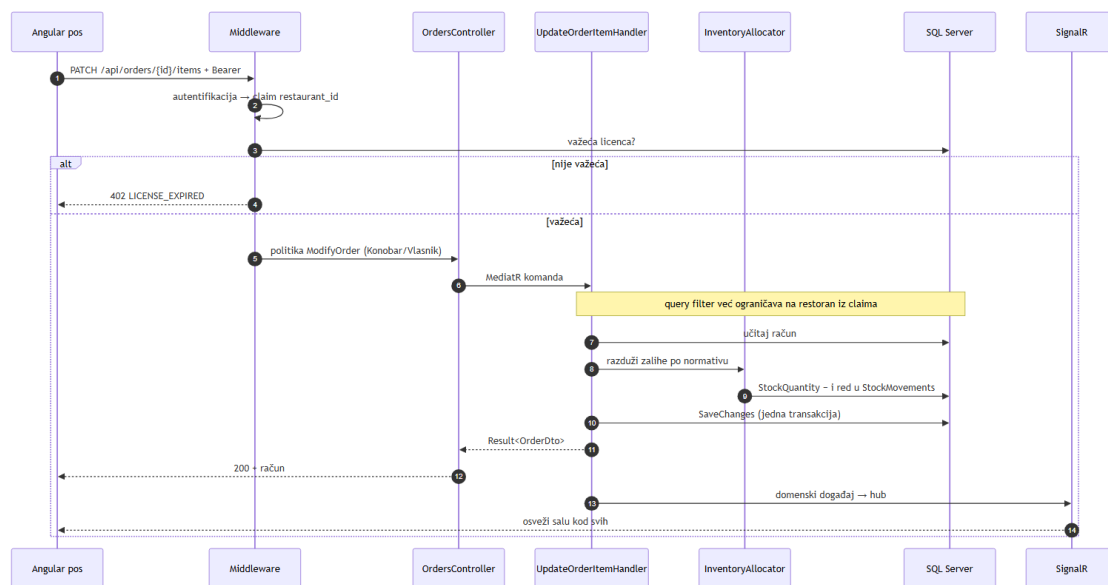
Раздвајање је појачано и тиме што два домаћина издају жетоне са различитим циљним примаоцем (енгл. audience). Жетон касе на административном домаћину не важи, и обрнуто, па ни украден жетон конобара не отвара ништа на платформској страни.

5.4. Пут једног захтева

Да би се слојеви видели у дејству, користан је један конкретан пример. У наставку је описано шта се дешава када конобар дода пиво на отворени рачун.

15. Клијент шаље захтев са жетоном у заглављу.
16. Међуслој проверава потпис жетона и из њега извлачи ознаку објекта.
17. Међуслој проверава да ли објекат има важећу лиценцу; ако нема, одговара кодом 402 и захтев даље не иде.
18. Контролер проверава овлашћење према матрици из поглавља 7 и прослеђује команду посреднику.

19. Обрађивач учитава рачун, при чему га глобални филтер већ ограничава на објекат из жетона.
20. Обрађивач раздужује залихе по нормативу артикла.
21. Измена рачуна и промена залиха уписују се у истој јединици посла, па или прођу заједно или никако.
22. Клијент добија измењен рачун у одговору.
23. Доменски догађај се прослеђује каналу за комуникацију у реалном времену, који осталим клијентима јавља да освеже салу.



Слика 2. Пут једног захтева: додавање ставке на отворени рачун

Исти ток приказан је и на слици 2, у облику дијаграма секвенце. Гранање на њој одговара трећем кораку претходног списка, то јест провери лиценце, после које захтев или пролази даље или се прекида.

Три ствари овај пут држе заједно. Лиценца се проверава пре контролера, па је ниједна крајња тачка не може заборавити; залихе и рачун се снимају у истом позиву, па не могу разићи; а комуникација у реалном времену је последица, а не замена за одговор, јер клијент који је послао измену добија је кроз сам одговор.

5.5. Комуникација у реалном времену

Екран сале не би био употребљив да се мора освежавати ручно, јер у објекту истовремено ради више људи над истим столовима. За освежавање је коришћена библиотека SignalR [7], која преко трајне везе шаље обавештења свим отвореним екранима.

Обавештења намерно носе врло мало података — у суштини само поруку да се нешто променило — а екран затим поново прочита стање. Реконструисање стања из низа измена било би брже, али је и уобичајен начин да два конобара виде различит укупан износ за исти сто, док је један додатни захтев над базом овог обима занемарљив.

Постоје два канала, један за кухињу и један за салу, јер немају исту публику. Кухињу не занима да је гост стигао на резервацију, а салу не занима да је јело почело да се припрема, па се раздвајањем избегава да сваки екран обрађује догађаје који га се не тичу.

6. Модел података

Шема базе није цртана унапред него је изведена из доменског модела, миграцијама. Ово поглавље описује њен облик, а пре свега механизам којим се подаци различитих објеката држе раздвојени, јер је то одлука са најдалекосежнијим последицама у целом раду.

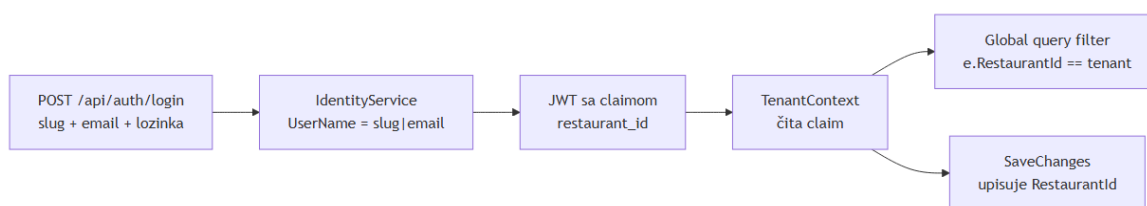
6.1. Раздвајање закупаца

Постоје три уобичајена начина да се у систему са више закупаца раздвоје подаци: засебна база по закупцу, засебна шема по закупцу, и једна база са колоном која именује закупца. У овом раду изабран је трећи начин, јер прва два захтевају да се при свакој промени шеме миграција примени на све базе, што би у раду овог обима било тешко и показати и одржавати.

Пресудно је, међутим, како је тај трећи начин изведен. Свака табела која припада објекту носи колону са ознаком објекта, али се та колона не појављује ни у једном упиту који пишу обрађивачи: контекст базе је поставља при упису и по њој одсеца читање, кроз глобални филтер над упитима.

Разлика је суштинска. Када би услов стајао у упиту, сигурност система зависила би од тога да ниједан од неколико десетина упита није заборављен; овако зависи од једне класе, а ред туђег објекта за обрађивача једноставно не постоји.

Ознака објекта увек долази из потписаног жетона, никада из путање, заглавља или тела захтева. Да није тако, измена једног поља у захтеву била би довољна да се види туђи објекат, што је вероватно најчешћа грешка у системима овог типа.



Слика 3. Пут ознаке објекта од пријаве до филтера над упитима

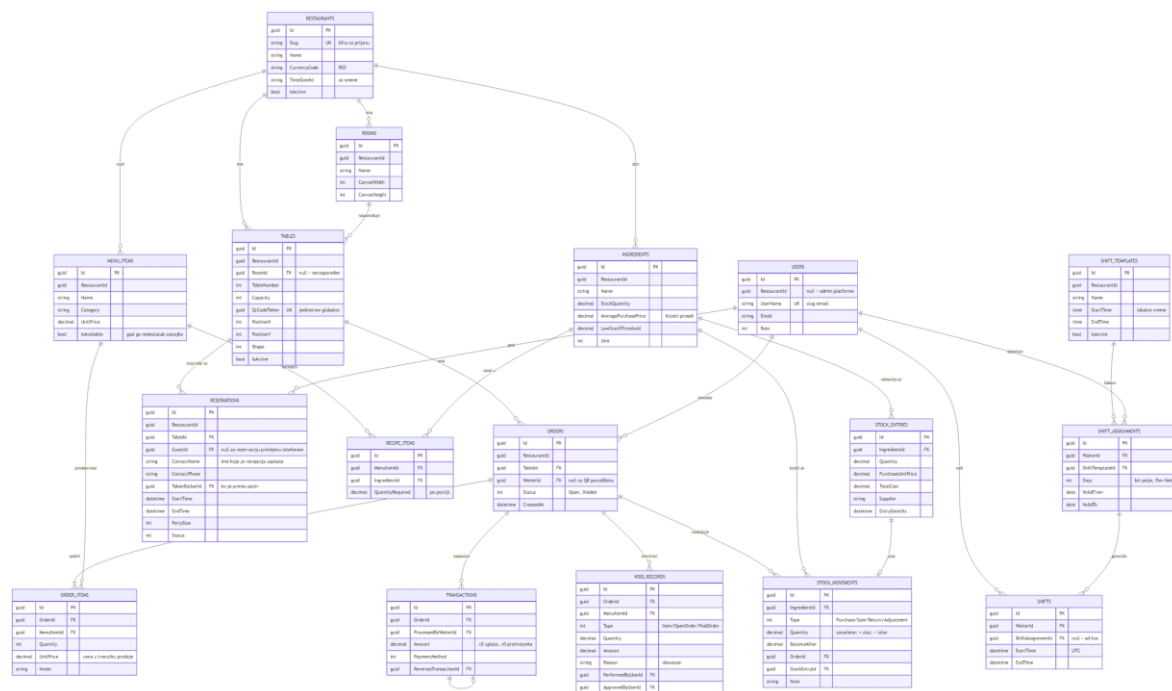
Слика 3 приказује цео пут те ознаке, од пријаве до филтера. Битно је да на том путу не постоји ниједна тачка на којој би клијент могао да је измени, јер се она чита искључиво из потписаног жетона.

Одлука има и цену, коју треба навести поштено. Колона са ознаком објекта на тим табелама нема страни кључ ка табели објеката, па интегритет држи код а не база; грешка у коду могла би уписати ред са погрешним објектом, а база не би имала шта да каже.

6.2. Пословни део модела

Средиште модела је рачун, који повезује сто, конобара и ставке. Око њега стоје јеловник са нормативима, магацин са састојцима и књигом промета, и евиденција наплате.

Везе између њих приказане су на слици 4. Дијаграм је изведен из истог доменског модела из ког су настале и миграције, па одговара стварној шеми базе.



Слика 4. Модел података: пословни део шеме

Слика 4 показује облик шеме и везе међу табелама, док појединачна поља на њој служе више као потврда да су ту него као текст за читање. Оно што сваки ентитет значи носи табела 5, која их именује редом којим податак настаје: од објекта и његове сале, преко рачуна, до наплате.

Ентитет	Улога у моделу
Објекат	Закупац; носи шифру за пријаву, валуту, временску зону и изабрану тему.
Просторија и сто	Распоред сале; сто носи број, капацитет, положај и токен за

Ентитет	Улога у моделу
	код.
Елемент просторије	Шанк, тоалет, улаз и слично; црта се, али није сто.
Рачун и ставка рачуна	Отворени или затворени рачун и његове ставке са запамћеном ценом.
Артикал јеловника	Назив, категорија, цена и доступност.
Састојак и норматив	Залиха по јединици мере и количина састојка по артиклу.
Улаз робе и књига промета	Набавка са ценом и сваки појединачни покрет залихе.
Трансакција	Наплата или противставка; противставка носи негативан износ.
Запис о сторнирању	Шта је сторнирано, колико, ко је то урадио и из ког разлога.
Корисник, смена и шаблон смене	Запослени и њихов распоред рада.
Резервација	Сто, време, број гостију и контакт.
Лиценца и уплата	Право објекта да ради и евиденција плаћања тог права.

Табела 5. Основни ентитети модела података

Природни кључеви који се понављају између објеката јединствени су тек у пару са ознаком објекта. Број стола, назив артикла, назив састојка и назив просторије могу се, дакле, понављати у различитим објектима, али не унутар истог.

Један изузетак од тог правила заслужује објашњење. Токен стола, који стоји у коду за скенирање, јединствен је на нивоу целе платформе, јер гост скенира код пре него што се зна о ком је објекту реч, па токен мора сам да разреши објекат.

6.3. Новац као вредносни објекат

Износи се у систему не преносе као обични бројеви него кроз мали вредносни објекат (енгл. value object) који сам заокружује и сабира. Разлог је тај што је заокруживање на две децимале одлука коју треба донети једном, а не на сваком месту на ком се сабира.

У бази се износи чувају као децимални тип, а не као број у покретном зарезу. За систем који сабира цене и прави збирове то није ствар прецизности у строгом смислу, него избегавања ситуације у којој се збир на екрану и збир на отиску разликују за један динар без видљивог разлога.

6.4. Записи који се не бришу

Две табеле у моделу су намерно дописиве само на крај: књига промета залиха и евиденција сторнирања. Ни један ни други запис се не мења нити брише, него се, када треба нешто поништити, додаје нови ред који поништава претходни.

Исти принцип примењен је и на наплату. Сторнирање плаћеног рачуна не брише трансакцију него уписује противставку са негативним износом, па збир свих трансакција и даље даје тачан промет, а историја остаје читљива.

Цена тог избора је нешто већи број редова и потреба да се извештаји пишу тако да противставке не броје као рачуне. Добитак је да ниједна радња у систему не уклања траг, што је за евиденцију која се тиче новца вероватно важније од уштеде простора.

7. Безбедност и контрола приступа

Безбедност је у овом систему изведена кроз четири одвојена механизма, који се преклапају. Преклапање је намерно, јер сваки од њих хвата другачију врсту грешке.

7.1. Аутентификација

Пријава тражи три податка: шифру објекта, адресу електронске поште и лозинку. Шифра објекта је нужна зато што иста адреса електронске поште може постојати у више објеката, па се корисничко име интерно чува као спој шифре објекта и адресе.

После успешне пријаве издаје се жетон у облику JSON веб жетона (енгл. JSON Web Token) [10], који носи ознаку корисника, његову улогу и ознаку објекта. Жетон је потписан, па се његов садржај не може изменити а да потпис остане исправан, и то је разлог због ког се ознака објекта узима искључиво из њега.

7.2. Матрица овлашћења

Контрола приступа заснована је на улогама (енгл. role-based access control) [12] и описана је на једном месту, као скуп именованих политика са списком улога које их задовољавају. Контролери се позивају на имена политика, тако да се матрица може прочитати у целини из једног фајла и упоредити са спецификацијом.

Неколико редова те матрице уже је него што би се очекивало, и то је намерно. Конобар не може да откаже резервацију, менаџер не може ни да отвори рачун ни да прими поруџбину преко кода са стола, а финансијске извештаје види искључиво власник.

Клијентска апликација пресликава исту матрицу у својим чуварима рута. То није безбедносна мера — сервер остаје једини ауторитет — него мера употребљивости, којом се избегава да се кориснику понуди екран који би сваки његов захтев одбио.

7.3. Сесија стола

Гост који скенира код са свог стола добија жетон који је намерно одвојен од жетона запослених. Тај жетон не доказује ко је гост — о госту се ништа и не зна — него искључиво за којим столом телефон седи.

Одвајање има две практичне последице. Гост који скенира код на телефону запосленог не може да потисне његову сесију, а истекла сесија стола не може никога да одјави из касе.

Сесија стола траје три сата и чува се у меморији картице прегледача, па нестаје када се картица затвори. Гост који је отишао не треба сутра и даље да поручује за сто број пет, а токен који остане у историји прегледача после тога не отвара ништа.

7.4. Лиценцни зид

Право објекта да користи систем проверено је у међуслоју, пре него што захтев стигне до контролера. Објекат без важеће лиценце добија одговор са статусним кодом 402, који означава да је плаћање неопходно, на сваки позив осим на онај којим се пита зашто.

Изузетак за питање о лиценци је битан: објекат који не може да користи касу и даље мора моћи да сазна разлог. Без тог изузетка корисник би видео само низ неуспелих захтева без икаквог објашњења, што би вероватно било протумачено као квар.

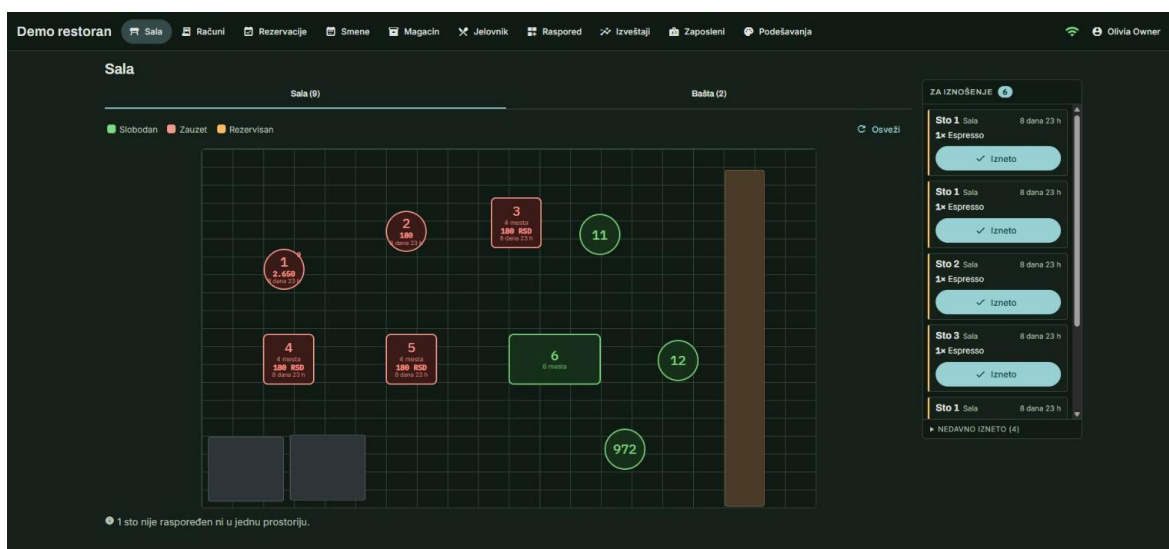
Клијентска апликација на тај код реагује тако што запосленог води на екран о лиценци, али госта не. Екран о лиценци именује објекат и упућује да се позове администратор платформе, што госту за столом не значи ништа, па његов екран уместо тога каже да објекат тренутно не прима поруџбине.

8. Кориснички интерфејс

Интерфејс касе разликује се од уобичајеног пословног интерфејса по томе што се не чита него се у њега погледа. Конобар који прелази салу подигне поглед и тражи један сто, па је мерило доброг екрана овде брзина препознавања, а не количина приказаних података.

8.1. Екран сале

Главни екран приказује распоред столова онако како су распоређени у просторији, а не као листу. Стање стола носи боја — слободан, заузет, резервисан — док се износ отвореног рачуна и време од отварања исписују у самом столу, тако да се за основну информацију не мора ништа отварати.



Слика 5. Екран сале, са распоредом столова и редом поруџбина са десне стране

На слици 5 види се да стање стола носе и боја и оквир, а не само боја. То је последица провере контраста из одељка 8.4, јер боја која сама носи значење не помаже особи која је не разликује.

Сала се сама уклапа у висину прозора, уместо да се прозор прилагођава сали. Просторија која излази испод доње ивице претворила би поглед у листање, а тражени сто био би управо онај испод прегипа, па се расположива висина мери при сваком исцртавању и цртеж се скалира да у њу стане.

Уз распоред стоји и ред поруџбина које су гости послали са својих телефона. Картице у њему поређане су од најстарије ка најновијој, јер је то ред којим се износе,

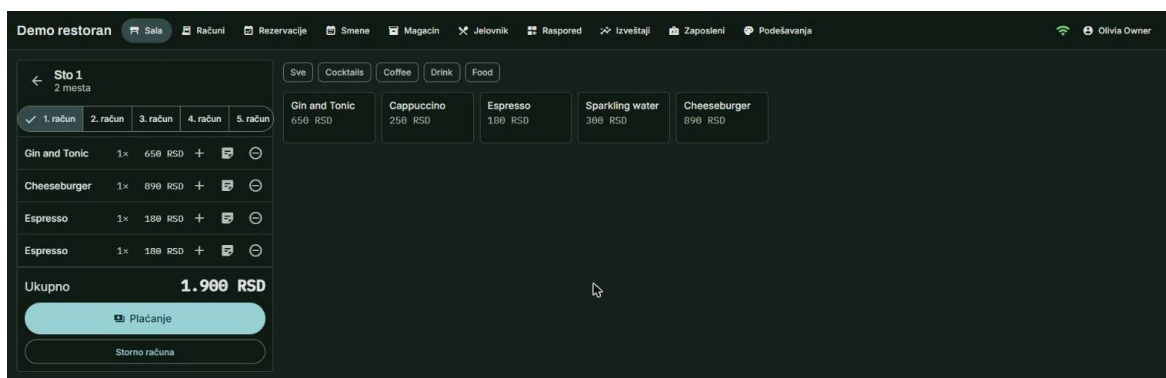
а не од најновије, како би се очекивало по угледу на обавештења у другим апликацијама.

8.2. Поништавање и звучни сигнал

Ред се у пракси ради једном руком, док се другом носи послужавник, па је погрешан притисак реалан а не хипотетички случај. Због тога се последњих неколико изнетих рунди задржава пола сата у засебној листи, одакле се могу вратити у ред.

Картица која се уклони не нестаје одмах него се скупи у току кратке анимације, па се картице испод ње помере навише уместо да поскоче у празнину. На екрану који се гледа летимично, поскакивање се тешко разликује од погрешног притиска, па је и то део истог разлога.

Картица се појављује сама, али само ако неко у том тренутку гледа у екран, што у бучном објекту по правилу није случај. Зато долазак нове поруџбине прати кратак звучни сигнал од два тона, који се може искључити у подешавањима, и то по уређају, јер таблет у сали и рачунар у канцеларији нису у истој просторији и немају разлога да се слажу.



Слика 6. Екран рачуна: отворен рачун стола и јеловник са десне стране

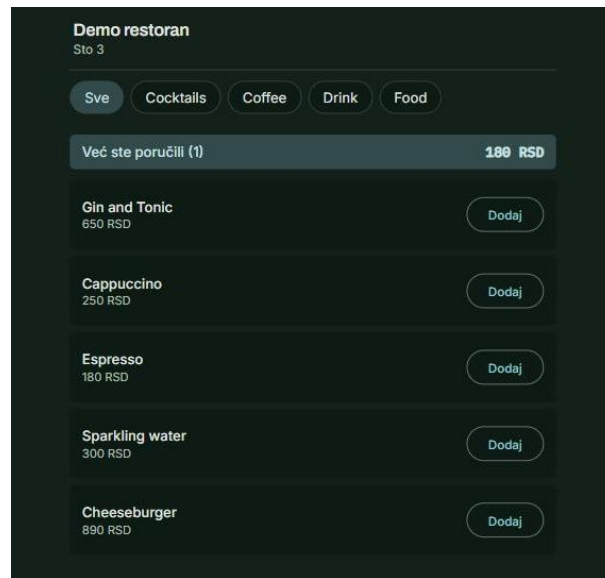
Екран рачуна, приказан на слици 6, држи отворен рачун са леве стране а јеловник са десне. Картице изнад ставки одговарају појединачним рундама, јер један сто може истовремено носити више отворених рачуна.

8.3. Гостов екран

Екран до ког води код са стола направљен је за телефон који се држи у једној руци: једна колона, велике мете за прст и корпа прикачена за доњу ивицу, где палац

дохвата. Нема ни пријаве ни налога, јер токен из кода представља целу сесију и говори само о ком је столу реч.

Гост не види рачун који би могао да плати, јер поручивање на овај начин и даље завршава тако што конобар донесе рачун. Оно што гост добија јесте јеловник објекта, начин да поруџбину пошаље без чекања да буде примећен, и текући списак онога што је сто до тада попио и појео.



Слика 7. Гостов екран, до ког води код са стола

Заглавље на слици 7 носи назив објекта и број стола, а боје су боје тог објекта. Гост тако види да је стигао тамо где је и мислио, што је једина потврда коју има, будући да се нигде не пријављује.

Тај текући списак не може се извести ни из једне појединачне поруџбине, јер свака рунда отвара свој рачун, па га сервер сабира по столу из жетона сесије. То је и једини податак који гостов екран тражи а који није могао доћи из онога што је сам послао.

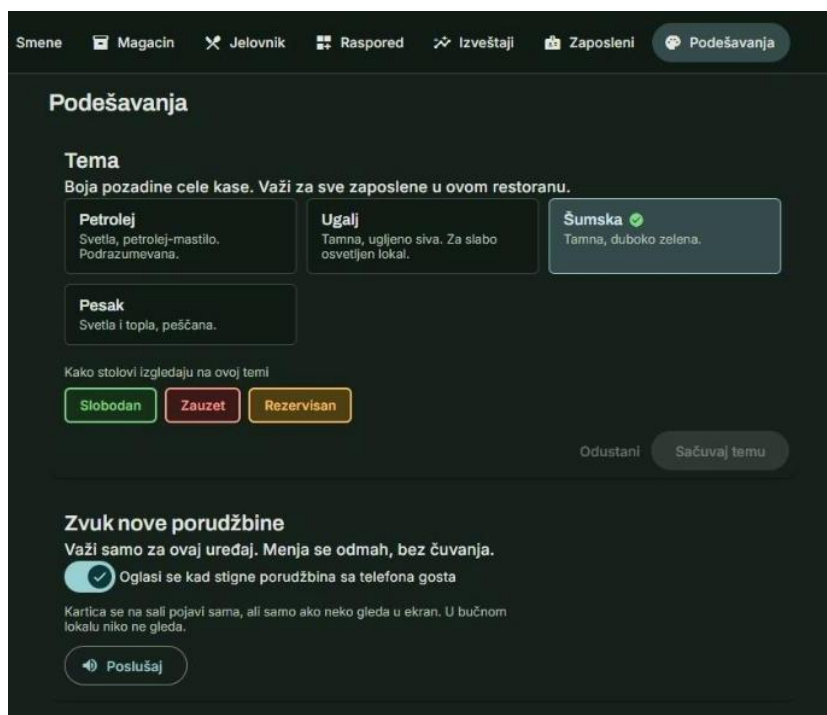
8.4. Теме

Власник бира палету за свој објекат, из затвореног скупа од четири теме. Скуп је намерно затворен, а не слободан бирач боје, јер на екрану сале боја значи стање, па би слободан избор пре или касније дао шанк у нијанси заузетог стола.

Тема	Подлога	Намена
Петролеј	Светла, плавозелена	Подразумевана.
Угаљ	Тамно сива	Тамни режим, за слабо осветљен објекат.

Тема	Подлога	Намена
Шумска	Тамно зелена	Тамна алтернатива угљу.
Песак	Светла, топла	Топао тон без тамне подлоге.

Табела 6. Понуђене теме и њихова намена



Слика 8. Избор теме, са приказом стања стола на изабраној подлози

Испод избора теме на слици 8 приказана су три стања стола на изабраној подлози. То није украс него провера: та три стања чине цео речник екрана сале, и управо их промена подлоге може учинити нечитљивим.

Пешчана тема из табеле 6 намерно је светла, иако је замишљена као „браон“. Црвена за заузет сто и наранџаста за резервисан по тону су најближе браон боји од свега у апликацији, па би тамно браон подлога била једини случај у ком би се боје стања почеле стапати са оквиром екрана.

Тема припада објекту, а не кориснику, јер је то одлука коју власник доноси за цео локал. Екран за пријаву, међутим, још не зна о ком је објекту реч, па се последња виђена тема памти на уређају и примењује пре првог исцртавања, док је одговор сервера исправља ако се не слажу.

Исти механизам користи и гостов екран, али без памћења: палета објекта се примени чим сесија стола буде успостављена, а на телефон госта се ништа не уписује. Гост тако види боје просторије у којој седи, док његов телефон после obroka не носи никакав траг објекта.

8.5. Величина првог учитавања

Заједничка библиотека је у првобитном облику била један модул, што је имало неочекивану последицу. Обе апликације увозе из ње пре него што се зна ко гледа, а алат за паковање дели код по изворном фајлу, па се испред екрана за пријаву нашао и цео слој за комуникацију у реалном времену и цео слој дијалога.

Поделом библиотеке на три улазне тачке — основну, компоненте и комуникацију у реалном времену — са почетног пакета касе скинуто је око 224 килобајта, а са административне апликације око 161 килобајт. Мерење је извршено поређењем извештаја алата за паковање пре и после промене.

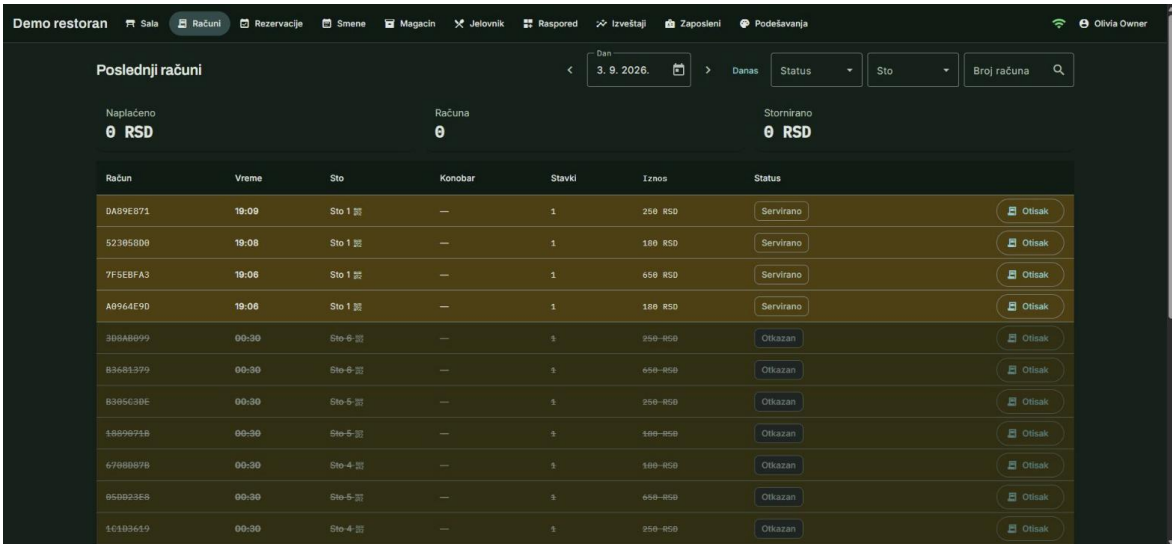
9. Извештавање

Извештаји су једини део система који власник гледа а не користи у току смене. Због тога су писани тако да свака бројка на екрану има јасно одређено значење и да ниједна не ласка.

9.1. Начело: ниједан извештај не ласка

Сва три финансијска извештаја обрачуната су тако да су сторна већ одбијена. Промет приказује оно што је објекат заиста задржао, а списак најпродаванијих артикала броји само оно што је заиста плаћено, па се не може надувати укуцавањем и накнадним сторнирањем.

Износ сторнираног приказује се одвојено, иако је већ одбијен. Разлог је тај што дан са великим сторном тако сам себе објашњава, уместо да само изгледа лоше.



Račun	Vreme	Sto	Konobar	Stavki	Iznos	Status
DAB9EB71	19:09	Sto 1	—	1	259 RSD	Servirano
52385809	19:08	Sto 1	—	1	189 RSD	Servirano
7F5EBFA3	19:08	Sto 1	—	1	659 RSD	Servirano
A0964E9D	19:08	Sto 1	—	1	189 RSD	Servirano
306AB099	00:30	Sto 6	—	1	259 RSD	Otkazan
83681399	00:30	Sto 6	—	1	659 RSD	Otkazan
838563DE	00:30	Sto 6	—	1	259 RSD	Otkazan
16879758	00:30	Sto 5	—	1	189 RSD	Otkazan
67088078	00:30	Sto 4	—	1	189 RSD	Otkazan
850023E8	00:30	Sto 5	—	1	659 RSD	Otkazan
16183619	00:30	Sto 4	—	1	259 RSD	Otkazan

Слика 9. Преглед последњих рачуна, са прецртаним отказаним редовима

Исто начело види се и на прегледу последњих рачуна, приказаном на слици 9. Отказани рачуни се не уклањају из списка него се прецртавају, па остају видљиви као траг уместо да нестану без објашњења.

9.2. Промет по данима

Промет се групише по локалним радним данима објекта, а не по данима у координираном светском времену. Рачун наплаћен у пола један после поноћи припада ноћи која га је произвела, а груписање по светском времену разбацало би касни сервис на два датума.

Због тога се груписање не може препустити бази него се изводи након учитавања, јер припадност локалном дану зависи од временске зоне објекта и њених правила о летњем рачунању времена. За количине података које један објекат произведе у години то није приметан трошак.

9.3. Сторнирање

Извештај о сторнирању постоји због једног обрасца који се не види нигде другде. Сторно је најлакши пут којим новац излази из касе, а конобар који сторнира знатно више од колега примети се тек када се појединачни записи саберу по запосленом.

Извештај приказује и појединачне записе, са разлогом који је уписан при сторнирању. Ти разлози се за сада не групишу, о чему је више речено у поглављу 11.

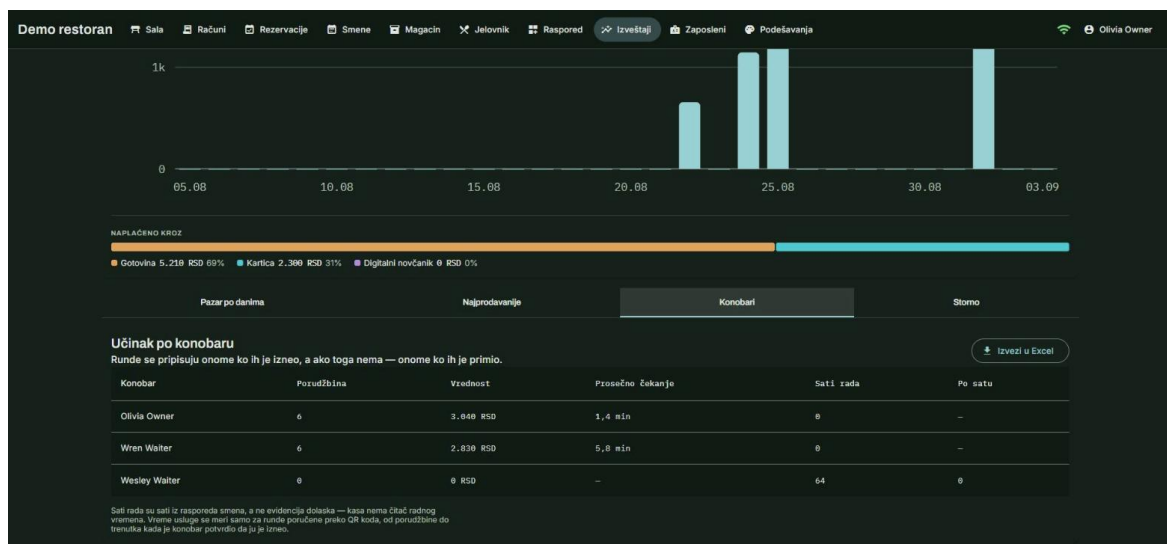
9.4. Учинак по конобару

Овај извештај је једини у систему који мери услугу а не новац, и настао је из запажања да у бази већ стоји податак који нико не гледа. Уз сваку поруџбину бележи се тренутак када је примљена и тренутак када је изнесена, па је разлика између њих једина бројка у систему која описује колико се чекало.

Да би се та разлика могла приписати особи, у модел је додат податак о томе ко је поруџбину изнео. За поруџбину коју је примио конобар та два чина се поклапају, али поруџбина послата са телефона госта нема конобара који ју је примио, па би без тог податка једини мерљиви чин услуге у целом систему припадао никоме.

Колона	Значење
Конобар	Име и презиме запосленог; један ред по особи.
Поруџбина	Број рунди приписаних тој особи, сторнираних и поништених изузев.
Вредност	Збир вредности тих рунди, по ценама запамћеним на ставкама.
Просечно чекање	Средње време од пријема до изношења, у минутима.
Мерено рунди	Број рунди на којима тај просек почива.
Сати рада	Сати из распореда смена који падају унутар изабраног периода.
По сату	Вредност подељена сатима рада, када их има.

Табела 7. Колоне извештаја о учинку по конобару



Слика 10. Извештај о учинку по конобару, са дугметом за извоз у табеларни формат

Изглед извештаја приказан је на слици 10. Испод табеле стоји напомена о томе шта сати рада заиста јесу, јер та бројка сама по себи не открива да потиче из распореда а не из евиденције доласка.

Колона „Мерено рунди“ из табеле 7 постоји зато што просек без броја мерења није тврдња него утисак. Просек преко две рунде и просек преко двеста рунди различите су ствари, па се број мерења приказује уз сам просек.

Сати рада су сати из распореда, а не евиденција доласка, јер систем нема читач радног времена. То ограничење је изричито исписано и на екрану и у извезеном документу, јер би представљање распореда као присуства било нетачно на начин који се не види из саме бројке.

Рунде се приписују ономе ко их је изнео, а ако тога нема, ономе ко их је примио. Сторниране и поништене рунде се не броје, чиме се спречава да се учинак повећа укупцавањем и накнадним брисањем.

9.5. Извоз у табеларни формат

Извештај о учинку по конобару може се преузети и као табеларни документ у формату Office Open XML [13], који отварају сви распрострањени програми за рад са табелама. Документ се саставља на серверу, из истог упита из ког се црта и табела на екрану, па се садржај датотеке и садржај екрана не могу разићи.

У документу се све бројке уписују као бројеви, а не као унапред обликован текст. Смисао предавања табеле уместо отиска јесте у томе да је прималац може сортирати,

сабрати и пренети у сопствену табелу, што низ текстуалних поља која личе на износе тихо онемогућава.

Просек у збирном реду израчунат је као пондерисани, а не као просек просека. Просек просека изједначио би конобара са две мерене рунде и конобара са осамдесет, па је сваки појединачни просек помножен бројем рунди на којима почива.

Коначно, назив датотеке носи период који извештај покрива. Два извоза различитих недеља која заврше у истом директоријуму под истим именом у пракси су један извоз, па назив одређује сервер а не прегледач.

10. Тестирање и провера

Провера је у овом раду организована у четири нивоа. Подела није направљена унапред него је расла: последња два нивоа додата су тек пошто се показало да поједине грешке остају невидљиве за све што не разговара са правим сервером базе података.

10.1. Јединични тестови домена

Доменски слој нема ниједну спољашњу референцу, па се његова правила проверавају без базе, без веб сервера и без икаквих замена. Тестови на том нивоу проверавају ствари попут тога да се плаћен рачун не може изменити, да се рунда не може означити као изнесена двапут и да збир ставки користи цену запамћену при поручивању.

Брзина тог нивоа није споредна корист него услов да се уопште користи. Скуп од сто тридесет једног теста извршава се за око двеста милисекунди, па се покреће после сваке измене, а не само пред предају.

10.2. Тестови апликацијског слоја

Обрађивачи се проверавају над контекстом базе у меморији, али над правим моделом, са свим кључевима, везама и конверзијама које систем користи у раду. Тако се избегава замена која би била једноставнија али би проверавала нешто друго од онога што се извршава.

На том нивоу проверавају се и правила која нису својина једног ентитета. Примери су генерисање распореда смена из сталних додела, исечак сати који падају унутар извештајног периода и приписивање рунде ономе ко ју је изнео.

10.3. Интеграциони тестови

Интеграциони тестови дижу цео серверски домаћин у истом процесу и шаљу му праве захтеве. На том нивоу проверавају се усмеравање, матрица овлашћења, лиценци зид и, пре свега, раздвајање закупаца, и то тако што се тест пријављује као један објекат и тражи запис другог.

10.4. Пролаз кроз живи систем

Последња два нивоа чине две скрипте које раде над покренутим серверима и правом базом. Прва пролази кроз сваку руту оба домаћина и проверава статусни код, а друга за сваку крајњу тачку која пише проверава шта је заиста уписано у базу.

Разлог за њихово постојање је конкретан. Провајдер базе у меморији не преводи језик упита у прави језик базе и не примењује ограничења, па су две раније нађене грешке биле невидљиве за све што не разговара са правим сервером, док је једна каснија — допуна залиха без реда у књизи промета — била невидљива за све што гледа само статусни код.

10.5. Резултати

Табела 8 даје преглед нивоа провере и броја тестова на сваком од њих у тренутку писања овог текста. Бројеви се са даљим радом мењају, па их треба схватити као снимак стања, а не као коначну вредност.

Ниво	Шта хвата	Број тестова
Јединични, домен	Доменска правила, прелазе стања, рачунање износа.	131
Јединични, апликација	Обрађиваче команди и упита над правим моделом.	62
Интеграциони	Усмеравање, овлашћења, лиценци зид, раздвајање закупаца.	34
Клијентски	Пресретаче, сесију стола, боје сале, обликовање текста.	101
Пролаз кроз руте	Сваку руту оба домаћина над живом базом.	скрипта
Пролаз кроз базу	Шта је свака крајња тачка заиста уписала.	скрипта

Табела 8. Нивои провере и број тестова

Укупан број аутоматизованих тестова у тренутку писања износи 328, од чега 227 на серверској и 101 на клијентској страни. Све наведене провере пролазе, али то, наравно, говори само о ономе што је провером обухваћено.

11. Критичка дискусија

Ово поглавље износи оно што у раду није добро решено, као и оно што је могло бити урађено другачије. Критика је подељена на ону која се односи на саму поставку рада и ону која се односи на стање у посматраној области.

11.1. Критика сопствене поставке

Најозбиљнији приговор односи се на изостанак фискализације. Систем који издаје отисак рачуна а није повезан са одобреним уређајем у Србији не може се користити за стварну продају, па је оно што је направљено, строго узев, потпун систем за све осим за онај део због ког каса и постоји.

Други приговор тиче се одлуке да колона са ознаком објекта нема страни кључ. Одлука је образложена у одељку 6.1 и вероватно је исправна, али значи да последња линија одбране од погрешног уписа није база него кôд, што је слабија гаранција него што би се могло пожелети за податке који се тичу новца.

Трећи приговор односи се на обим. Систем покрива врло широк скуп функција — од распореда столова до магацина — а свака од њих је због тога обрађена плитко; рад у ком би само једна од тих области била обрађена до краја вероватно би био убедљивији као истраживачки допринос, мада мање употребљив као производ.

Коначно, поруке о грешкама које сервер враћа написане су на енглеском, док је интерфејс на српском. То је недоследност коју корисник види у тренутку када му је најмање потребна, и последица је тога што су поруке писане у коду пре него што је одлучено да клијент има сопствени слој превода.

11.2. Критика стања у области

У прегледу из одељка 2.3 уочљиво је да се решења у овој области готово искључиво мере новцем. Промет, просечан рачун и најпродаванији артикал налазе се у сваком од њих, док се време које гост проведе чекајући, колико се могло утврдити из јавно доступних описа, ретко нуди као готова бројка.

То је утолико необичније што податак најчешће већ постоји. Сваки систем који бележи тренутак поруџбине и тренутак издавања има све што је потребно, па изгледа да је реч о изостанку интересовања, а не о техничкој препреци.

Друга примедба тиче се затворености података. Већина решења извоз нуди као додатну услугу или га не нуди уопште, чиме се подаци о пословању објекта држе у оквиру производа, што отежава промену добављача и вероватно није у интересу корисника.

11.3. Шта је могло другачије

Разлози сторнирања се уписују, али се не групишу. Када би се понудио списак честих разлога уз слободан унос, понављање разлога постало би мерљиво, а тиме и податак о томе где се новац губи; то није урађено јер је списак разлога ствар политике објекта, па би захтевао још један екран за подешавање.

Сто се не ослобађа сам, па рачун остаје отворен док се не наплати. Ако гости оду без плаћања или конобар заборави да затвори рачун, сто стоји заузет неограничено дуго, што тихо троши капацитет објекта; решење вероватно није аутоматско затварање, јер се новац не дира без човека, него тиха ознака да рачун траје неувобичајено дуго.

Коначно, конобар у великом објекту види цео ред поруџбина, укључујући и столове које покрива неко други. Филтер који би ред свео на његов део сале захтевао би да систем зна за појам дела сале, чега у моделу нема, па је та функција остала неуррађена иако изгледа једноставно.

12. Правци даљег рада

Идеје које следе наведене су без обавезивања да ће бити реализоване. Поређане су према односу очекиване користи и процењеног труда, почев од оних код којих је највећи део механизма већ направљен.

12.1. Екран за кухињу

Канал за комуникацију са кухињом већ постоји, догађаји о новој поруџбини и измени ставке се већ емитују, а прелаз у стање припреме већ је део доменског модела — само ниједан екран то не користи. Екран који би приказивао шта је поручено, шта је у припреми и шта је готово вероватно би био најјефтинија значајна допуна система.

12.2. Делови сале и распоред по њима

Увођење појма дела сале омогућило би филтер „само моји столови“ из одељка 11.3, али и нешто корисније од тога. Распоред смена могао би се правити по просторијама, па би се видело који део објекта у којој смени остаје непокривен.

12.3. Више валута и пореских стопа

Модел већ носи ознаку валуте објекта, али се она нигде не користи, јер је динар уписан у све шаблоне. То поље данас обећава нешто што систем не ради, па би га требало или искористити или уклонити, а прво од тога отворило би и питање пореских стопа, које су ионако неопходне за било какву озбиљну примену.

12.4. Рад без везе са интернетом

Каса која престане да ради када падне веза озбиљно је ограничење за објекат у ком се ради свакога дана. Решење би вероватно подразумевало локалну базу на уређају и разрешавање сукоба при поновном повезивању, што је посао знатно већег обима од осталих ставки на овом списку.

12.5. Анализа времена чекања по сатима и данима

Извештај о учинку по конобару мери просечно чекање по особи, али не и по времену. Исти податак груписан по сату и дану у недељи показао би власнику када

објекат не стиже, што је информација на основу које се мења распоред смена, а не оцена запосленог.

13. Закључак

У раду је пројектован и имплементиран веб систем за пословање угоститељског објекта у ком једна инсталација опслужује више међусобно одвојених објеката. Систем покрива отварање и наплату рачуна, јеловник, магацин, резервације, распоред смена, поручивање од стране госта и извештавање, уз засебну апликацију за регистрацију објеката и вођење лиценци.

Основна архитектонска одлука била је да пословна правила остану у слоју без спољашњих зависности, а раздвајање података различитих објеката да буде својство контекста приступа бази. Прва одлука учинила је да се доменска правила проверавају за око двеста милисекунди, а друга је могућност цурења података свела са неколико десетина упита на једну класу.

Као садржински допринос уведена је мерена величина која описује услугу а не промет: време од пријема до изношења поруџбине, приписано запосленом и извезено у табеларни формат. Да би та величина уопште могла да се припише особи, модел је морао да забележи и ко је поруџбину изнео, што је измена мале величине али без које једини мерљиви чин услуге у систему не би припадао никоме.

Исправност је проверавана на четири нивоа, при чему су последња два настала као одговор на конкретне грешке невидљиве за тестове који не разговарају са правим сервером базе података. То искуство вероватно је и најпреносивији налаз рада: ниво провере који не додирује праву инфраструктуру не може посведочити о понашању које та инфраструктура одређује.

Најозбиљније ограничење рада је изостанак фискализације, због ког систем у садашњем облику није употребљив за стварну продају у Србији. Уклањање тог ограничења, заједно са екраном за кухињу чији је највећи део механизма већ направљен, чини се као природан следећи корак.

Литература

- [1] Evans, E., „Domain-Driven Design: Tackling Complexity in the Heart of Software“, Addison-Wesley, 2003.
- [2] Martin, R. C., „Clean Architecture: A Craftsman’s Guide to Software Structure and Design“, Prentice Hall, 2017.
- [3] Fowler, M., „Patterns of Enterprise Application Architecture“, Addison-Wesley, 2002.
- [4] Fowler, M., „CQRS“, martinowler.com, 2011, <https://martinfowler.com/bliki/CQRS.html>
- [5] Microsoft, „.NET documentation“, Microsoft Learn, <https://learn.microsoft.com/dotnet/>
- [6] Microsoft, „Entity Framework Core documentation“, Microsoft Learn, <https://learn.microsoft.com/ef/core/>
- [7] Microsoft, „Real-time ASP.NET Core with SignalR“, Microsoft Learn, <https://learn.microsoft.com/aspnet/core/signalr/>
- [8] Angular Team, „Angular documentation“, <https://angular.dev/>
- [9] Nottingham, M., Wilde, E., „RFC 7807: Problem Details for HTTP APIs“, IETF, 2016, <https://www.rfc-editor.org/rfc/rfc7807>
- [10] Jones, M., Bradley, J., Sakimura, N., „RFC 7519: JSON Web Token (JWT)“, IETF, 2015, <https://www.rfc-editor.org/rfc/rfc7519>
- [11] „Закон о фискализацији“, Службени гласник Републике Србије, бр. 153/2020, са каснијим изменама.
- [12] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., Youman, C. E., „Role-Based Access Control Models“, IEEE Computer, vol. 29, br. 2, 1996, DOI: 10.1109/2.485845.
- [13] ISO/IEC 29500, „Information technology — Document description and processing languages — Office Open XML File Formats“, ISO/IEC, 2016.
- [14] Nielsen, J., „Usability Engineering“, Morgan Kaufmann, 1993.
- [15] W3C, „Web Content Accessibility Guidelines (WCAG) 2.1“, W3C Recommendation, 2018, <https://www.w3.org/TR/WCAG21/>

Напомена за довршавање: уз веб изворе треба дописати датум приступа, а уз књиге број ISBN, у складу са смерницама. Списак треба допунити изворима коришћеним у поглављу 2, чим преглед постојећих решења буде проширен конкретним производима.

Додатак А: изводи из програмског кода

У овом додатку сабрани су изводи из кода на које се текст позива, издвојени овде да не би прекидали излагање. Коментари у коду су изворни, онакви какви стоје у пројекту, а испод сваког извода дат је кратак опис на српском језику.

А.1. Филтер закупца над упитима

Следећи извод је срце механизма описаног у одељку 6.1. Прва метода проналази све типове ентитета који припадају објекту, а друга сваком од њих додаје филтер.

```
/// <summary>Adds the tenant filter for one entity type.</summary>
/// <remarks>
/// The predicate reads _tenantContext.RestaurantId through the context
/// instance rather than capturing the value. EF Core compiles the model
/// once and reuses it for the lifetime of the application, so a captured
/// value would freeze whichever restaurant happened to sign in first and
/// hand its data to everybody else. Read through the instance, it becomes
/// a query parameter evaluated per request.
/// </remarks>
private void ApplyRestaurantFilter<TEntity>(ModelBuilder builder)
    where TEntity : class, IRestaurantScoped
{
    builder.Entity<TEntity>()
        .HasQueryFilter(entity =>
            entity.RestaurantId == _tenantContext.RestaurantId);
}
```

Извод 1. Постављање филтера закупца за један тип ентитета

Детаљ наведен у коментару извода 1 није ситница. Алат за мапирање модел гради једном и користи га до краја рада апликације, па би запамћена вредност замрзнула објекат који се први пријавио и његове податке предала свима осталима.

Уз читање, исти контекст брине и о упису. Метода из извода 2 поставља ознаку објекта на новоубачене редове, али само тамо где она већ није попуњена.

```
private void ApplyRestaurantStamp()
{
    if (!_tenantContext.HasTenant)
    {
        return;
    }

    foreach (EntityEntry<IRestaurantScoped> entry in
        ChangeTracker.Entries<IRestaurantScoped>())
    {
        if (entry.State == EntityState.Added
            && entry.Entity.RestaurantId == Guid.Empty)
        {
            entry.Entity.RestaurantId = _tenantContext.RestaurantId;
        }
    }
}
```

```

    }
  }
}

```

Извод 2. Уписивање ознаке објекта на нове редове

Услов да вредност већ није постављена постоји због два случаја. Агрегат који је своју ознаку већ проследио својој деци задржава ту вредност, а административна апликација и даље може да упише ред у име именованог објекта.

А.2. Доменско правило: изношење рунде

Извод 3 показује како изгледа доменско правило у овом систему. Метода не прима никакав објекат из инфраструктуре и не зна ништа о бази, па се може проверити без иједне спољашње зависности.

```

/// <summary>Records that the round went out, when, and who took it.</summary>
/// <param name="servedAtUtc">The instant it reached the table.</param>
/// <param name="servedByWaiterId">
/// Whoever pressed the button, or null where the caller is not staff.
/// </param>
public void MarkServed(DateTime servedAtUtc, Guid? servedByWaiterId = null)
{
    TransitionTo(OrderStatus.Served, OrderStatus.Open,
                OrderStatus.InPreparation);
    ServedAtUtc = servedAtUtc;
    ServedByWaiterId = servedByWaiterId;
}

/// <summary>
/// Puts a round back on the queue after it was marked carried out.
/// </summary>
public void ReopenForService()
{
    TransitionTo(OrderStatus.Open, OrderStatus.Served);
    ServedAtUtc = null;
    ServedByWaiterId = null;
}

```

Извод 3. Прелазак рунде у стање „изнето“ и повратак из њега

Позив методе за прелазак стања наводи дозвољена полазна стања, па се рунда не може означити као изнесена из било ког стања. Повратак је могућ искључиво из стања „изнето“, чиме се спречава да се плаћена или отказана рунда врати у ред.

А.3. Приписивање рунде и исечак сати

Извод 4 садржи два правила извештаја описаног у одељку 9.4. Прво приписује рунду особи, а друго од смене узима само онај део који пада унутар изабраног периода.

```
// Whoever carried it out, falling back to whoever took it. On a
// waiter's own tab those are the same act; on a guest QR round
// only the first exists.
.Select(order => new Round(
    order.ServedByWaiterId ?? order.WaiterId,
    order.CreatedAt,
    order.ServedAtUtc,
    order.OrderItems.Sum(item =>
        (decimal?)(item.UnitPrice * item.Quantity)) ?? 0m))

// Overlapping rather than contained: a shift that starts before the
// period or runs past its end still contributes the hours inside it.
var hours = shifts.Sum(shift =>
    (Min(shift.EndUtc, toUtc) - Max(shift.StartUtc, fromUtc)).TotalHours);
```

Извод 4. Приписивање рунде конобару и исечак распореда на период извештаја

Прво правило је разлог због ког је у модел уопште додат податак о томе ко је рунду изнео. Без њега би рунда послата са телефона госта остала без иједне особе уз себе, јер је поље о конобару на таквој рунди по дефиницији празно.

A.4. Звучни сигнал

Сигнал описан у одељку 8.2 није снимак него се синтетише у прегледачу. Извод 5 приказује срж тог поступка, са два тона и постепеним порастом и падом јачине.

```
OrderAlertService.NOTES.forEach((frequency, index) => {
    const startsAt = context.currentTime
    + index * OrderAlertService.NOTE_SECONDS;
    const endsAt = startsAt + OrderAlertService.NOTE_SECONDS;

    const oscillator = context.createOscillator();
    const gain = context.createGain();

    oscillator.type = 'sine';
    oscillator.frequency.value = frequency;

    // Ramped rather than switched. A gain that jumps to its value clicks, and a
    // click in a quiet room is the part people ask to have turned off.
    gain.gain.setValueAtTime(0.0001, startsAt);
    gain.gain.exponentialRampToValueAtTime(0.2, startsAt + 0.02);
    gain.gain.exponentialRampToValueAtTime(0.0001, endsAt);

    oscillator.connect(gain).connect(context.destination);
    oscillator.start(startsAt);
    oscillator.stop(endsAt);
});
```

Извод 5. Синтеза звучног сигнала за нову поруџбину

Два тона удаљена су за квинту, јер се интервал боље пробија кроз жамор него једна висина и не звучи као пријава грешке. Синтеза уместо снимка значи и да нема датотеке која се мора преузети пре него што први сигнал може да се огласи.

A.5. Препознавање нове поруџбине у реду

Сигнал се оглашава само за рунду која раније није виђена. Извод 6 показује како се то разликовање изводи, и зашто скуп виђених рунди обухвата и оне које су управо изнесене.

```
const arrived = panel.waiting.some((ticket) => !this.seen.has(ticket.id));

this.seen.clear();

for (const ticket of [...panel.waiting, ...panel.recentlyServed]) {
  this.seen.add(ticket.id);
}

// Not on the first read. A waiter opening the floor screen to four
// waiting rounds is being told what is already on the board, not
// that something just came in.
if (this.hasQueue && arrived) {
  this.alert.play();
}

this.hasQueue = true;
```

Извод 6. Разликовање пристигле рунде од рунде враћене у ред

Да скуп обухвата само рунде које чекају, картица враћена у ред после погрешног притиска огласила би сигнал као да је стигла нова поруџбина. Услов о првом читању спречава да отварање екрана сале огласи оно што на табли већ стоји.